

SUDOKU SOLVER USING BACKTRACKING AND RECURSION

MINI PROJECT REPORT

Submitted By

- K. Arshiya
- K. Gayathri

Department

Computer Science – Data Science

Academic Year

2023–2027

ABSTRACT

The Sudoku Solver is a Python-based application that automatically solves Sudoku puzzles using the Backtracking Algorithm and Recursion. Sudoku is a logic-based puzzle where numbers must be placed in a 9×9 grid while satisfying specific constraints. The solver systematically fills empty cells by trying valid numbers and uses recursion to explore possible solutions. If an invalid state is reached, the algorithm backtracks and tries alternative values until a valid solution is found. This project demonstrates the practical application of recursion and backtracking in solving constraint satisfaction problems.

INTRODUCTION

Sudoku is one of the most popular logic puzzles. Solving complex Sudoku puzzles manually can be difficult and time-consuming.

Backtracking is a powerful algorithmic technique used to solve problems by exploring all possible solutions. Combined with recursion, it efficiently solves Sudoku puzzles by testing possible values and reverting incorrect choices when necessary.

PROBLEM STATEMENT

Manual Sudoku solving requires careful analysis and can be slow for difficult puzzles.

The project aims to develop an automated Sudoku solver that:

- Solves any valid Sudoku puzzle.
 - Reduces solving time.
 - Demonstrates recursive problem-solving techniques.
 - Applies backtracking to satisfy Sudoku constraints.
-

OBJECTIVES

- Automatically solve Sudoku puzzles.
 - Implement Backtracking Algorithm.
 - Demonstrate Recursion.
 - Validate Sudoku rules.
 - Produce the completed puzzle efficiently.
-

METHODOLOGY

Step 1

Represent the Sudoku puzzle as a 9×9 matrix.

Step 2

Find the next empty cell.

Step 3

Try numbers from 1 to 9.

Step 4

Check whether the number satisfies:

- Row constraint
- Column constraint
- 3×3 subgrid constraint

Step 5

If valid, place the number.

Step 6

Recursively solve the remaining puzzle.

Step 7

If no valid number exists, backtrack and try another value.

Step 8

Repeat until the puzzle is solved.

ALGORITHM

1. Start
 2. Find an empty cell.
 3. Try numbers 1–9.
 4. Check validity.
 5. Place valid number.
 6. Call recursion.
 7. If failure occurs, remove number.
 8. Try next number.
 9. Continue until solved.
 10. End.
-

SYSTEM FLOWCHART

Start



Input Sudoku Puzzle



Find Empty Cell



Try Numbers 1–9



Check Sudoku Rules



Valid?



Yes → Place Number → Recursive Call



No → Try Next Number



Backtrack if Required



Puzzle Solved



Display Solution



End

IMPLEMENTATION

The project is implemented in Python.

Technologies Used

- Python
- Backtracking Algorithm
- Recursion

- 2D Arrays
-

RESULTS AND OUTPUT

The Sudoku Solver successfully solves valid Sudoku puzzles.

Sample Input

A partially filled 9×9 Sudoku grid.

Sample Output

A completely solved Sudoku puzzle satisfying all Sudoku rules.

ADVANTAGES

- Solves puzzles automatically.
 - Demonstrates recursion concepts.
 - Guarantees a correct solution if one exists.
 - Efficient for standard Sudoku puzzles.
 - Easy to understand and implement.
-

LIMITATIONS

- Performance decreases for highly complex variants.
 - Does not generate Sudoku puzzles.
 - Solves only valid Sudoku grids.
-

FUTURE SCOPE

- Graphical User Interface (GUI)
 - Mobile application
 - AI-assisted Sudoku solving
 - Image recognition using Computer Vision
 - Sudoku puzzle generator
 - Difficulty level detection
-

CONCLUSION

The Sudoku Solver Using Backtracking and Recursion efficiently solves Sudoku puzzles by exploring possible solutions recursively and backtracking whenever necessary. The project demonstrates how recursion and backtracking can solve real-world constraint satisfaction problems and serves as an excellent example of algorithmic problem solving.

REFERENCES

1. Python Official Documentation.
2. Introduction to Algorithms – Cormen et al.
3. Backtracking Algorithm Tutorials.
4. Sudoku Solving Research Papers.
5. Data Structures and Algorithms textbooks.